

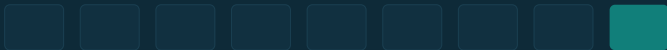


STREAM PROCESSING AVEC APACHE KAFKA

Premiers pas en Java

Échauffement Java, puis les API Producer et Consumer

ESGI · 5^e année · Data Engineering · 2025–2026



Objectifs de la séance

- **Mettre en place un projet Java / Maven pour Kafka**

Une seule dépendance : `kafka-clients`.

- **(Ré)activer les idiomes Java utiles**

`record`, `var`, `lambdas`, `try-with-resources`.

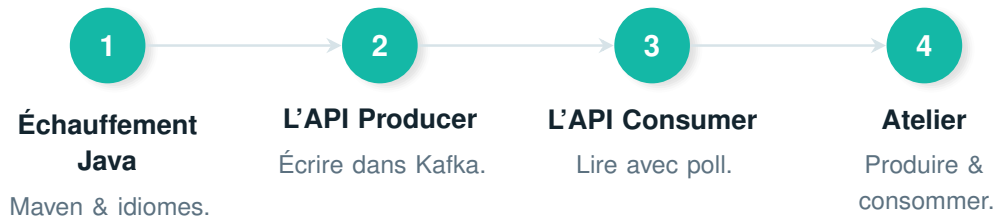
- **Écrire un producteur en Java**

Configurer, créer un `KafkaProducer`, envoyer des messages.

- **Écrire un consommateur en Java**

S'abonner et lire dans une boucle `poll`.

Au programme



01

- PARTIE 1

Échauffement Java & projet Maven

Pourquoi Java pour ce cours ?

- Le client officiel est natif JVM ; les API riches — Kafka Streams, Interactive Queries, `TopologyTestDriver` — n'existent qu'en Java.
- Les clients Java demandent Java 11+ ; on utilise **Java 17 ou 21** (records, `var`, `switch` moderne).
- **Maven** gère le projet et les dépendances ; tout le TP du cours se fait en Java.
- Les langages JVM (Kotlin, Scala) marchent aussi, mais on reste en Java pour la cohérence.

Objectif aujourd'hui : être à l'aise avec un projet minimal avant d'attaquer Kafka Streams.

Maven : une seule dépendance

Un projet Maven standard. La dépendance `kafka-clients` fournit producteur, consommateur et (de)sérialiseurs.

`pom.xml` (extrait)

```
<properties>
  <maven.compiler.release>21</maven.compiler.release>
</properties>

<dependencies>
  <dependency>
    <groupId>org.apache.kafka</groupId>
    <artifactId>kafka-clients</artifactId>
    <version>4.3.0</version>
  </dependency>
</dependencies>
```

Les idiomes Java qu'on va utiliser

Java

```
// record : porteur de donnees immuable et concis  
record Commande(String client, int montant) {}  
  
// var : le type est infere par le compilateur  
var c = new Commande("client-1", 42);  
  
// lambda : un comportement passe en argument  
liste.forEach(x -> System.out.println(x));  
  
// try-with-resources : fermeture automatique du bloc  
try (var producer = new KafkaProducer<String, String>(props)) {  
    // ... producer.close() est appele en sortie de bloc  
}
```

02

- PARTIE 2

L'API Producer

Anatomie d'un producteur



On configure, on crée le producteur, puis on lui confie des enregistrements à envoyer.

Un producteur minimal

ProducerApp.java

```
// config du producteur
var props = new Properties();
props.put("bootstrap.servers", "localhost:9092");
props.put("key.serializer", StringSerializer.class.getName());
props.put("value.serializer", StringSerializer.class.getName());
props.put("acks", "all");

// envoi de 10 messages avec une cle
try (var producer = new KafkaProducer<String, String>(props)) {
    for (int i = 0; i < 10; i++) {
        var record = new ProducerRecord<>(
            "commandes", "client-" + i, "montant=" + i);
        producer.send(record);
    }
}
```

L'envoi est asynchrone

- `send()` ne bloque pas : il rend un `Future`.
- Un *callback* reçoit le résultat : les métadonnées (partition, offset) ou une exception.
- `flush()` force l'envoi ; `close()` vide le tampon.
- `try-with-resources` s'en charge en fin de bloc.

callback

```
producer.send(record, (meta, ex) -> {  
    if (ex != null)  
        ex.printStackTrace();  
    else  
        System.out.printf("p=%d off=%d%n",  
                           meta.partition(), meta.offset());  
});
```

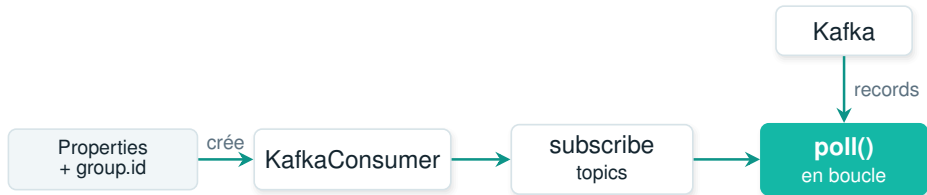
Kafka regroupe les envois par lots en arrière-plan : c'est ce qui rend le débit élevé.

03

- PARTIE 3

L'API Consumer

Anatomie d'un consommateur



On configure (avec un `group.id`), on s'abonne, puis on lit en boucle avec `poll`.

Un consommateur minimal

ConsumerApp.java

```
var props = new Properties();
props.put("bootstrap.servers", "localhost:9092");
props.put("key.deserializer", StringDeserializer.class.getName());
props.put("value.deserializer", StringDeserializer.class.getName());
props.put("group.id", "atelier-s4");
props.put("auto.offset.reset", "earliest");

try (var consumer = new KafkaConsumer<String, String>(props)) {
    consumer.subscribe(List.of("commandes"));
    while (true) {
        var records = consumer.poll(Duration.ofMillis(500));
        for (var r : records)
            System.out.printf("p=%d off=%d %s%n",
                               r.partition(), r.offset(), r.value());
    }
}
```

La boucle `poll`, et trois réglages clés

`poll(Duration)` renvoie un lot de messages (`ConsumerRecords`) ; on l'itère, puis on reboucle.

■ `group.id`

Le groupe de consommateurs qui se partagent le travail (détaillé en S5).

■ `auto.offset.reset`

Sans offset connu :
`earliest` (tout) ou
`latest` (nouveaux).

■ `commit des offsets`

Auto par défaut ; le commit manuel sera vu en S5.

L'offset commité est la mémoire du groupe : il sait où reprendre après un arrêt.

04

• PARTIE 4 · ATELIER

Produire & consommer en Java

Un starter Maven à compléter

- Projet Maven prêt, avec des `// TODO` à compléter
- Deux classes : `ProducerApp` et `ConsumerApp`
- Une solution de référence dans le package `solution`
- Objectif : produire 10 messages, puis les relire

arborescence

```
src/main/java/  
  app/ProducerApp.java    # TODO  
  app/ConsumerApp.java    # TODO  
  solution/                # corrige  
pom.xml
```

Du code qui écrit et qui lit

- 1 Ouvrir le projet Maven
- 2 Compléter `ProducerApp`
- 3 Lancer le producteur
- 4 Compléter `ConsumerApp`
- 5 Lancer le consommateur
- 6 Observer partition & offset

bash

```
# compiler le projet
mvn -q compile

# lancer le producteur
mvn -q exec:java \
    -Dexec.mainClass=app.ProducerApp

# lancer le consommateur
mvn -q exec:java \
    -Dexec.mainClass=app.ConsumerApp
```

Ce que le consommateur affiche

sortie console

```
p=0 off=12 montant=3  
p=2 off=8  montant=1  
p=1 off=5  montant=4  
p=0 off=13 montant=7  
...
```

■ Même group.id

Relancé, il reprend après le dernier offset
commité — rien à relire.

■ Nouveau group.id

Avec `earliest`, il relit tout le topic depuis le
début — le rejeu.

Récapitulatif

- ✓ Un projet Maven + `kafka-clients` suffit pour produire et consommer.
- ✓ Producteur : `Properties` → `KafkaProducer` → `send(ProducerRecord)`.
- ✓ `send()` est asynchrone ; un callback confirme partition et offset.
- ✓ Consommateur : `subscribe` puis boucle `poll(Duration)`.
- ✓ `group.id` et l'offset commité déterminent où l'on reprend.



LA SUITE — SÉANCE 5

Consumer groups & parallélisme

Comment plusieurs consommateurs se partagent les partitions, le rebalancing (et le nouveau protocole KIP-848), et le commit manuel des offsets.